

Frill: ML on Z80

A Functional Language for Vintage Hardware

MinZ Project

2026-03-23

- [Frill: ML on Z80](#)
 - [What Is Frill?](#)
 - [Why ML on Z80?](#)
 - [What Frill Has That C Doesn't](#)
 - [Frill vs ML-Family Languages](#)
 - [Language Tour](#)
 - [Functions and Let Bindings](#)
 - [Recursion](#)
 - [Pipe Operator and Composition](#)
 - [Algebraic Data Types and Pattern Matching](#)
 - [Parametric Polymorphism](#)
 - [Effect System](#)
 - [Property-Based Testing](#)
 - [While Loops and Mutation](#)
 - [String Operations](#)
 - [Let-In Inside Expressions](#)
 - [DSL Showcase](#)
 - [Expression Evaluator](#)
 - [Music DSL](#)
 - [Graphics Patterns](#)
 - [How It Compiles](#)
 - [Compile-Time Verification](#)
 - [Metrics](#)
 - [Getting Started](#)
 - [Appendix: Feature Reference](#)

Frill: ML on Z80

What Is Frill?

Frill is an ML-style functional language that compiles to Z80 machine code. The name is deliberately ironic: “frills” — decorations, embellishments — on the most constrained hardware imaginable. 64KB RAM. 3.5MHz. No OS.

And yet: algebraic data types, pattern matching, parametric polymorphism, an effect system, property-based testing, pipe operators, and lambda expressions. All compiling to the same tight Z80 assembly a hand-written program would produce.

“What if OCaml ran on a ZX Spectrum?”

Why ML on Z80?

This sounds absurd. ML-family languages (OCaml, Haskell, F#) are associated with garbage collectors, runtime systems, and megabytes of memory. Why bring these ideas to an 8-bit CPU from 1976?

Because the *ideas* are independent of the *implementation costs*.

ML Concept	The Idea	Z80 Reality
ADT	Model data as tagged variants	A u8 tag + payload. Same as C enum + union, but type-safe
Pattern matching	Dispatch on structure	Compiles to CP n / JR Z chains — same as hand-written
Parametric polymorphism	Write once, use with any type	Monomorphized at compile time. Zero runtime cost
Effect tracking	Know which functions do I/O	Pure functions are provably side-effect-free. Compiler enforces it
Pipe operator	Data flows left-to-right	Compiles to direct CALL — no overhead
Property testing	Verify for ALL inputs	Runs 256 tests at <i>compile time</i> . Bugs caught before the binary exists

The key insight: **Frill has no runtime system**. No GC, no closures allocated on heap, no vtables, no boxed values. Every abstraction compiles to the same code you’d write in assembly. The abstractions exist only at compile time.

What Frill Has That C Doesn’t

If you’re writing Z80 programs in C (via SDCC), you already have structs, functions, and loops. What does Frill add?

1. **Pattern matching** — Exhaustive, compiler-checked dispatch. No forgotten switch cases.
2. **Pipe operator** — `x |> f |> g` reads left-to-right. No nested `g(f(x))`.
3. **Compile-time verification** — `assert fib 7 == 13` runs during compilation.

4. **Effect system** — IO annotation prevents accidental side effects in pure code.
5. **Property testing** — `prop |x| add x 0 == x` tests all 256 u8 values.
6. **Polymorphism** — `let id (x : 'a) = x` works for u8 AND u16 without duplication.
7. **Type-safe ADTs** — `type Color = Red | Green | Blue` with exhaustive matching.

Frill vs ML-Family Languages

How close is Frill to “real” ML? Honest comparison:

Feature	OCaml/ F#	Haskell	Frill	Status
Let bindings	<code>let x = 5 in x + 1</code>	Same	Same	Identical syntax
ADT	<code>type t = A B of int</code>	<code>data T = A B Int</code>	<code>type T = A B of u8</code>	Works
Pattern matching	Full	Full	Match + guards	Works (no nested patterns yet)
Parametric polymorphism	<code>'a -> 'a</code>	<code>a -> a</code>	<code>'a -> 'a</code>	Works (monomorphized)
Type inference	Hindley-Milner	HM + extensions	Partial (params annotated)	Partial
Higher-order functions	First class	First class	Via pipes + compose	Works (no closures)
Pipe operator	<code>F#: ></code>	N/A	<code>></code>	Identical to F#
Effect system	N/A	Monads	IO annotation	Simpler than Haskell
Linear types	N/A	Linear Haskell	QTT (!, ~)	Works
Lazy evaluation	Option	Default	No	By design (strict)
Garbage collection	Yes	Yes	No	By design (no heap)

Feature	OCaml/ F#	Haskell	Frill	Status
Modules	Yes	Yes	import	Basic
Type classes	N/A	Yes	class/ instance	Partial
Closures	Heap- allocated	Heap- allocated	No (lambda = named function)	Limitation

What's missing: nested pattern matching, recursive ADTs (lists), closures that capture mutable state, full Hindley-Milner inference, modules with signatures.

What's unique to Frill: compiles to Z80 assembly, property testing at compile time, QTT linearity annotations, zero runtime overhead.

Language Tour

Functions and Let Bindings

```
(* Functions: name, typed parameters, body *)
let add (a : u8) (b : u8) = a + b
let double (x : u8) = x + x

(* Let-in bindings – scoped values *)
let hypotenuse_sq (x : u8) (y : u8) =
  let xx = x * x in
  let yy = y * y in
  xx + yy

(* Compile-time verification *)
assert add 3 5 == 8
assert double 7 == 14
assert hypotenuse_sq 3 4 == 25
```

Recursion

```
let factorial (n : u8) =
  if n == 0 then 1
  else n * factorial (n - 1)

let fib (n : u8) =
  if n < 2 then n
  else fib (n - 1) + fib (n - 2)
```

```
assert factorial 5 == 120
assert fib 7 == 13
```

Pipe Operator and Composition

```
let inc (x : u8) = x + 1
let dbl (x : u8) = x * 2
let sq (x : u8) = x * x

(* Pipe: data flows left to right *)
let transform (x : u8) = x |> dbl |> inc

(* Composition: combine functions *)
let dbl_then_inc = dbl >> inc

assert transform 5 == 11      (* 5*2+1 = 11 *)
assert dbl_then_inc 5 == 11
```

Algebraic Data Types and Pattern Matching

```
type Color = Red | Green | Blue

let color_name (c : u8) =
  match c with
  | Red   -> 82    (* 'R' *)
  | Green -> 71    (* 'G' *)
  | Blue  -> 66    (* 'B' *)

type Day = Mon | Tue | Wed | Thu | Fri | Sat | Sun

let is_weekend (d : u8) =
  match d with
  | Sat -> 1
  | Sun -> 1
  | _   -> 0

assert color_name Red == 82
assert is_weekend Sat == 1
assert is_weekend Mon == 0
```

Parametric Polymorphism

```
(* Type variables: 'a is resolved at each call site *)
let id (x : 'a) : 'a = x
let poly_max (a : 'a) (b : 'a) : 'a = if a > b then a else b
let poly_clamp (x : 'a) (lo : 'a) (hi : 'a) : 'a =
  if x < lo then lo else if x > hi then hi else x
```

```
(* Each call generates a specialized u8 version *)
assert id 42 == 42
assert poly_max 10 20 == 20
assert poly_clamp 50 10 200 == 50
assert poly_clamp 5 10 200 == 10
```

Under the hood, `id 42` generates `id_u8(x: u8) -> u8 = x`. Same as Rust's monomorphization. Zero runtime dispatch, optimal register usage.

Effect System

```
(* Pure function – no side effects, safe for compile-time eval *)
let add (a : u8) (b : u8) = a + b

(* IO function – explicitly marked, can do I/O *)
let greet (x : u8) : IO u8 =
  do puts "Hello!\r\n"
  0

(* Compiler enforces: pure cannot call IO *)
(* let bad (x : u8) = do puts "oops" -- ERROR! *)
```

The effect system is simpler than Haskell's monads. You mark functions : IO and the compiler checks that pure code stays pure. Extern functions (assembly, system calls) are implicitly IO.

Property-Based Testing

```
(* prop runs the predicate for ALL 256 u8 values *)
prop |x| add x 0 == x      (* identity *)
prop |x| poly_max x x == x (* idempotent *)
prop |x| id x == x        (* polymorphic identity *)
```

Each prop generates 256 compile-time assertions. If any value fails, the compiler reports it before the binary is even created. This is exhaustive testing for u8 — not sampling, not heuristics, every single input.

While Loops and Mutation

```
let sum_to (n : u8) =
  let acc = 0 in
  let i = 0 in
  while i < n do
    do acc <- acc + i
    do i <- i + 1
  end
  acc
```

```
assert sum_to 10 == 45
```

Frill supports imperative loops where needed. The `do var <- expr` syntax makes mutation explicit — you always see where state changes.

String Operations

```
let char_at (s : u16) (n : u8) = peek (s + n)
let str_len (s : u16) : u8 =
  let p = 0 in
  while peek (s + p) > 0 do
    do p <- p + 1
  end
  p

assert char_at "Hello" 0 == 72  (* 'H' *)
assert char_at "Hello" 4 == 111 (* 'o' *)
assert str_len "Hello" == 5
```

Let-In Inside Expressions

```
(* let-in works everywhere – even inside if/else branches *)
let classify (x : u8) =
  if x > 100 then
    let half = x / 2 in half
  else
    let doubled = x * 2 in doubled

assert classify 200 == 100
assert classify 50 == 100
```

DSL Showcase

Expression Evaluator

A complete recursive-descent parser and evaluator in Frill, with correct operator precedence and parentheses:

```
let eval (src : u16) : u8 = unpack_val (eval_expr src 0)

assert eval "5" == 5
assert eval "3+4" == 7
assert eval "3+4*2" == 11  (* precedence: * before + *)
assert eval "2*(3+4)" == 14 (* parentheses *)
assert eval "(1+2)*3" == 9
```

17 compile-time assertions verify the evaluator handles every case correctly. The parser is ~60 lines of Frill, demonstrating that functional style makes recursive descent natural and readable.

Music DSL

Notes as algebraic data types, frequencies via pattern matching:

```
type Note = C | Cs | D | Ds | E | F | Fs | G | Gs | A | As | B | Rest

let note_delay (n : u8) =
  match n with
  | C    -> 133    (* C4 = 262 Hz *)
  | E    -> 106    (* E4 = 330 Hz *)
  | G    -> 89     (* G4 = 392 Hz *)
  | A    -> 79     (* A4 = 440 Hz *)
  | Rest -> 0
  | ...

(* Melody encoding: note + duration packed into one byte *)
let encode (note : u8) (dur : u8) : u8 = note * 16 + dur

(* Verified: encode-decode round-trip *)
prop |x| decode_note (encode (x / 16) (x % 16)) == x / 16
```

33 asserts + 2 properties verify the entire music system at compile time.

Graphics Patterns

Pure functions that generate pixel data — Sierpinski triangles, XOR textures, checkerboards, smiley sprites:

```
(* Sierpinski: the classic bitwise fractal *)
let sierpinski (x : u8) (y : u8) =
  if (x & y) == 0 then 1 else 0

(* XOR texture: demoscene plasma effect *)
let xor_tex (x : u8) (y : u8) = (x ^ y) % 8

(* Compose into a multi-region display *)
let composite (x : u8) (y : u8) =
  if y < 32 then
    if x < 32 then sierpinski x y * 5
    else xor_tex x y
  else
    if checker x y 8 > 0 then 6 else 1
```


The pattern functions are pure and verified at compile time. The rendering loop runs on real Z80 hardware.

How It Compiles

Frill compiles through the full MinZ pipeline:

```
Frill (.frl) → HIR → MIR2 → Z80 Assembly → Binary
```

Each step is a standard compiler transformation: - **Frill → HIR**: Desugar ML syntax into a typed intermediate representation - **HIR → MIR2**: Lower to SSA form with block arguments - **MIR2 → Z80**: Register allocation (PBQP + WFC), instruction selection, peephole optimization

A simple function like `let add (a : u8) (b : u8) = a + b` compiles to:

```
add:
    ADD A, B      ; a + b (result in A)
    RET          ; return
```

Two instructions. Zero overhead compared to hand-written assembly.

Compile-Time Verification

Frill's assert and prop statements run during compilation on the MIR2 virtual machine. This means:

- **Bugs are caught before the binary exists**
- **No test framework needed** — the compiler IS the test runner
- **Exhaustive for u8** — prop tests all 256 values, not a sample
- **Cross-verified** — same code runs on VM and Z80, both must agree

Current stats: **13 example files, 3351 compile-time checks, 0 failures.**

Metrics

Metric	Value
Language features	41
Example programs	13
Compile-time checks	3,351
Parser LOC (frill.go)	2,436
Stdlib modules	5 (math, tokenizer, I/O, functional, canvas)

Metric	Value
Compilation target	Z80 (ZX Spectrum, CP/M, Agon Light 2)

Getting Started

```
# Compile a Frill program
minzc hello.frl -b z80 -o hello.a80

# Compile for CP/M
minzc hello.frl -b z80 -t cpm -o hello.com

# All assertions are checked automatically during compilation
```

Create a file hello.frl:

```
let double (x : u8) = x + x
let inc (x : u8) = x + 1

assert double 5 == 10
assert inc 99 == 100
assert 5 |> double |> inc == 11

let main (x : u8) = 0
```

If every assert passes, you get a Z80 binary. If any fails, the compiler tells you exactly which assertion and what it got vs what was expected.

Appendix: Feature Reference

Feature	Syntax	Example
Function	let name (p : type) = body	let inc (x : u8) = x + 1
If-else	if cond then a else b	if x > 0 then x else 0
Let-in	let x = e in body	let y = x * 2 in y + 1
While	while cond do ... end	while i < n do ... end
Mutation	do var <- expr	do i <- i + 1
Pipe	expr \> fn	5 \> double \> inc
Compose	f >> g	let h = inc >> double
ADT	type T = A \> B \> C	

Feature	Syntax	Example
		<pre>type Color = Red Green Blue</pre>
Match	<pre>match e with \ P -> v</pre>	<pre>match c with \ Red -> 2</pre>
Polymorphism	<pre>(x : 'a) : 'a</pre>	<pre>let id (x : 'a) : 'a = x</pre>
Effect	<pre>: IO type</pre>	<pre>let greet (x : u8) : IO u8 = ...</pre>
Assert	<pre>assert fn args == val</pre>	<pre>assert add 3 5 == 8</pre>
Property	<pre>prop \ x\ pred</pre>	<pre>prop \ x\ add x 0 == x</pre>
Peek	<pre>peek addr</pre>	<pre>peek (str + n)</pre>
Import	<pre>import "path"</pre>	<pre>import "../..//stdlib/frill/math.frl"</pre>
Extern	<pre>extern name (p : t) : t</pre>	<pre>extern putchar (ch : u8) : u8</pre>
Lambda	<pre>\ x\ body</pre>	<pre>5 \ > \ x\ x * 2</pre>
String	<pre>"text"</pre>	<pre>"Hello\r\n"</pre>
Linearity	<pre>(! x : t) / (~ x : t)</pre>	<pre>(! buf : u16) (use once)</pre>